



WEEK 1

SOFTWARE DEVELOPMENT TOOLS AND ENVIRONMENTS



Presented by **Asst. Prof. Dr. Tuchsanaï Ploysuwan**



- Git was developed in 2005 by **Linus Torvalds**
- Git is **Version control system** is a system that records changes to a file or set file over time so that you. can restore specific version later
- Git is a **Distributed Version Control System**







Git – What and Why

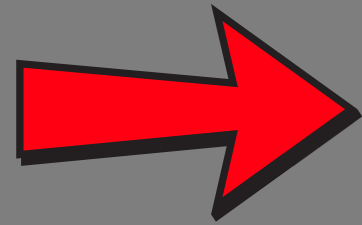
Before Version Control System



Before Version Control System



Before Version Control System



Before Version Control System



Version 1.0



Version 2.0

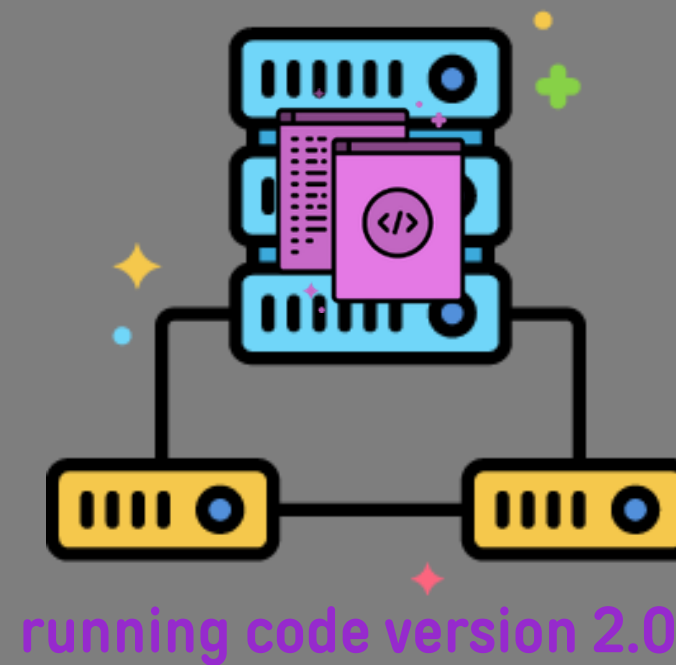
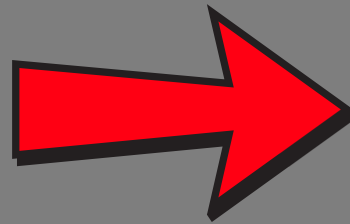


I have a great idea,
send me your code



running code version 1.0

Before Version Control System



Before Version Control System



Before Version Control System



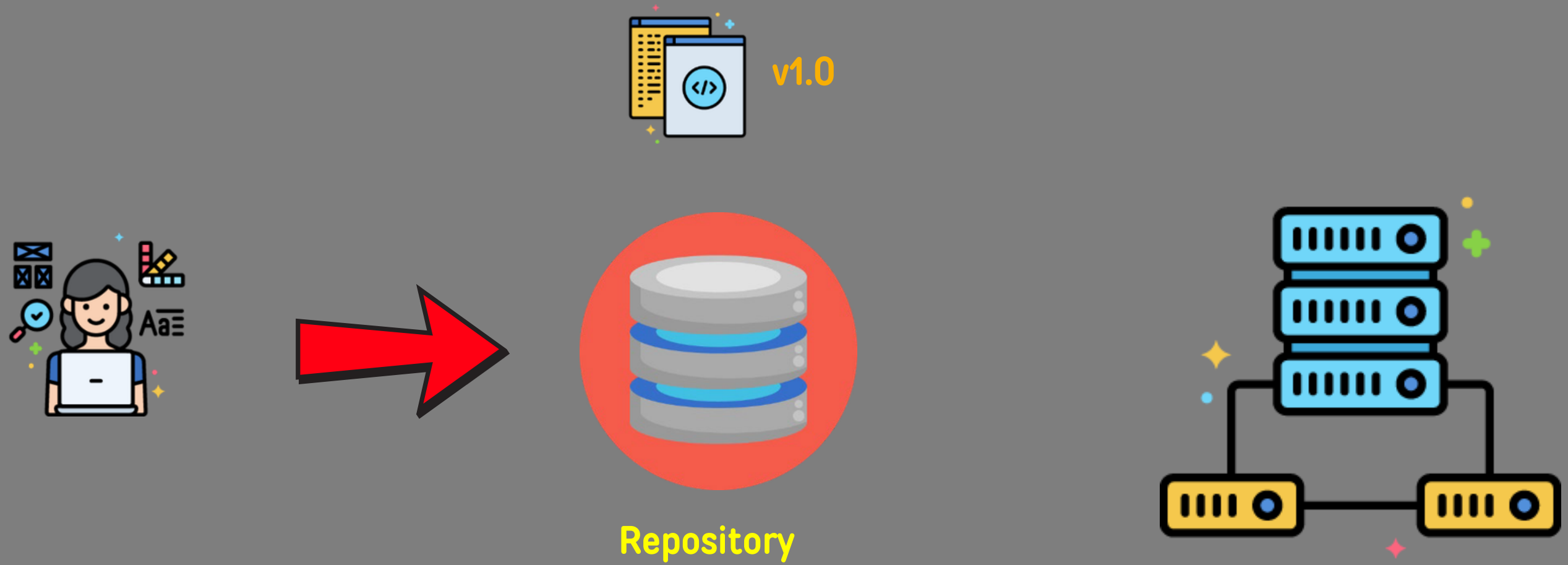
Version 2.0



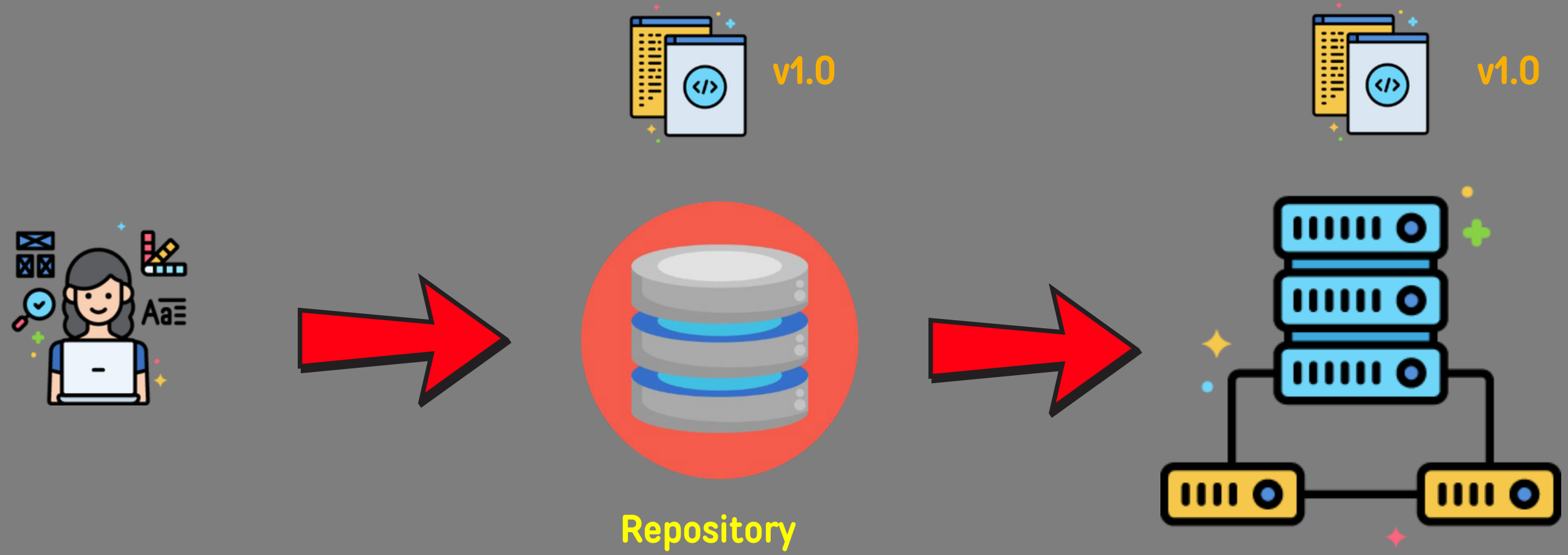
running code version 2.0

- Rollback is time consuming
- No audit tracking
- Not scalable for large teams

Version Control System



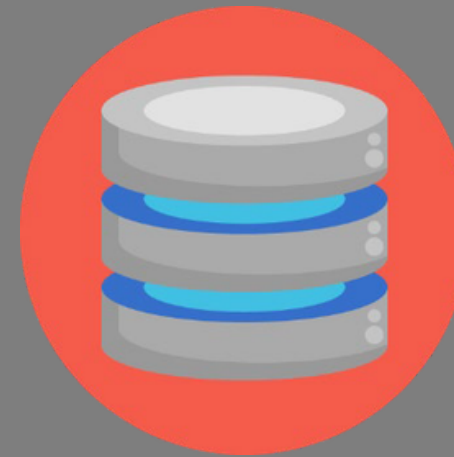
Version Control System



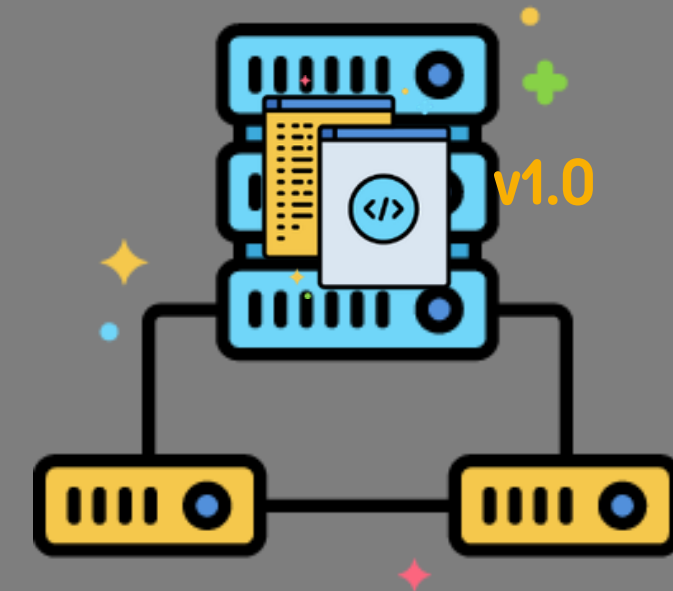
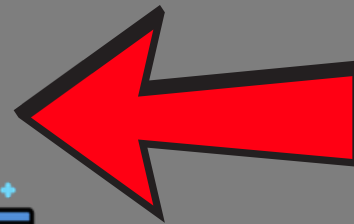
Version Control System



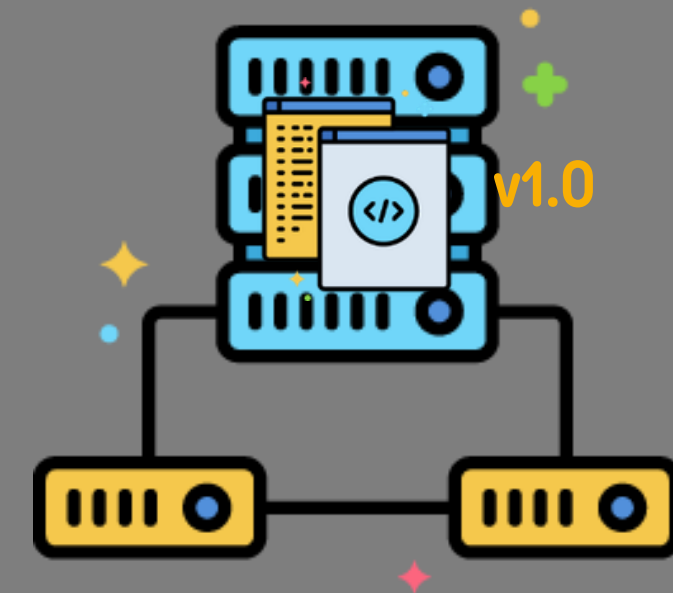
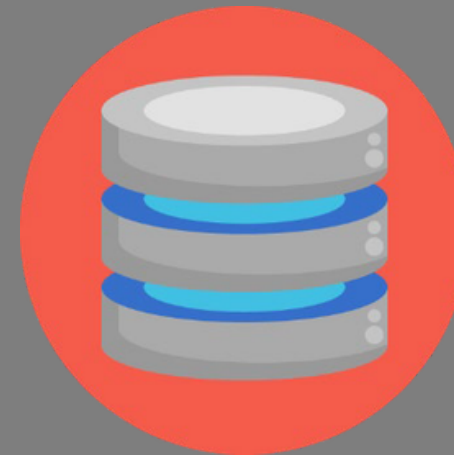
Fetch the code



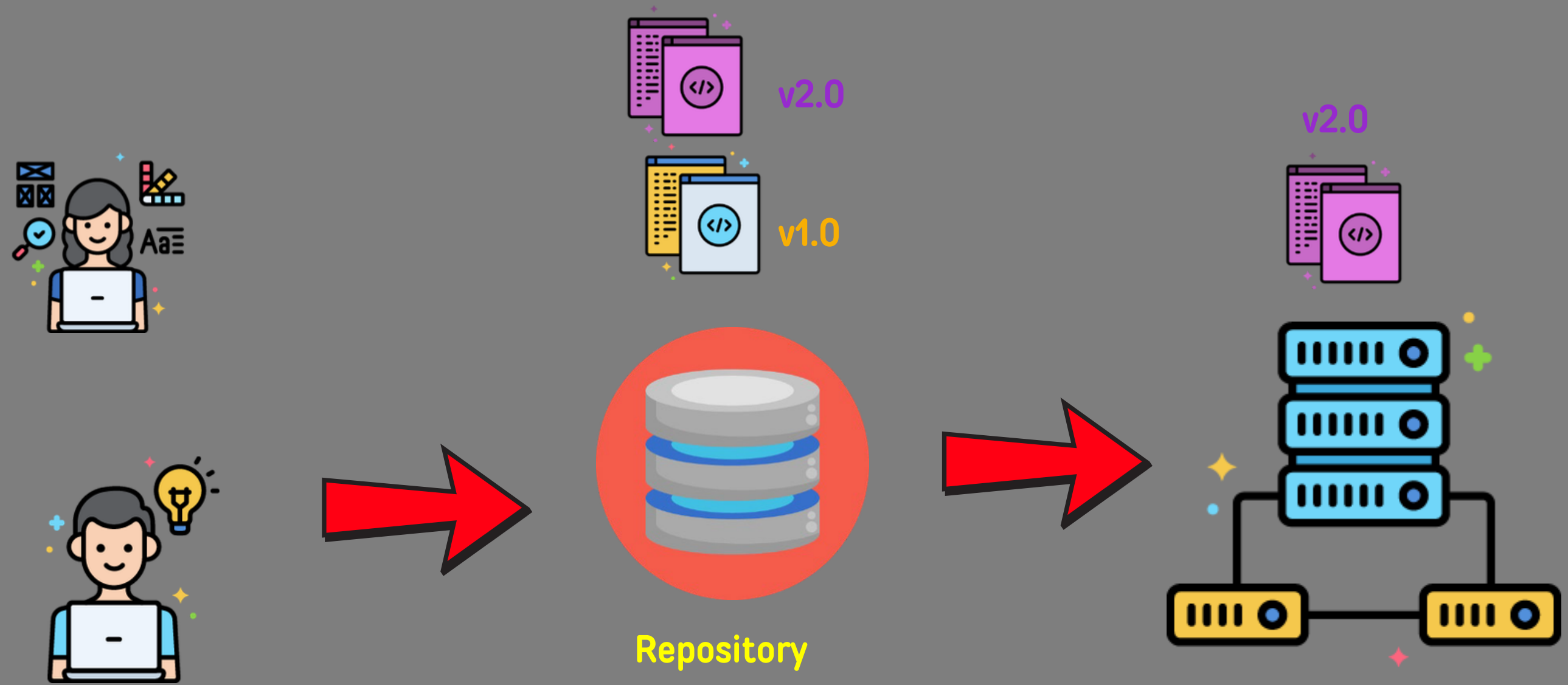
v1.0
Repository



Version Control System



Version Control System



Version Control System



Repository



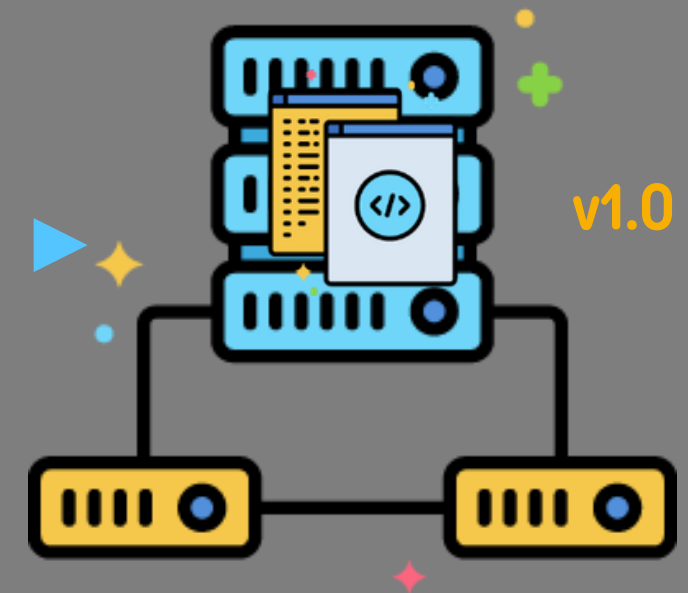
Version Control System



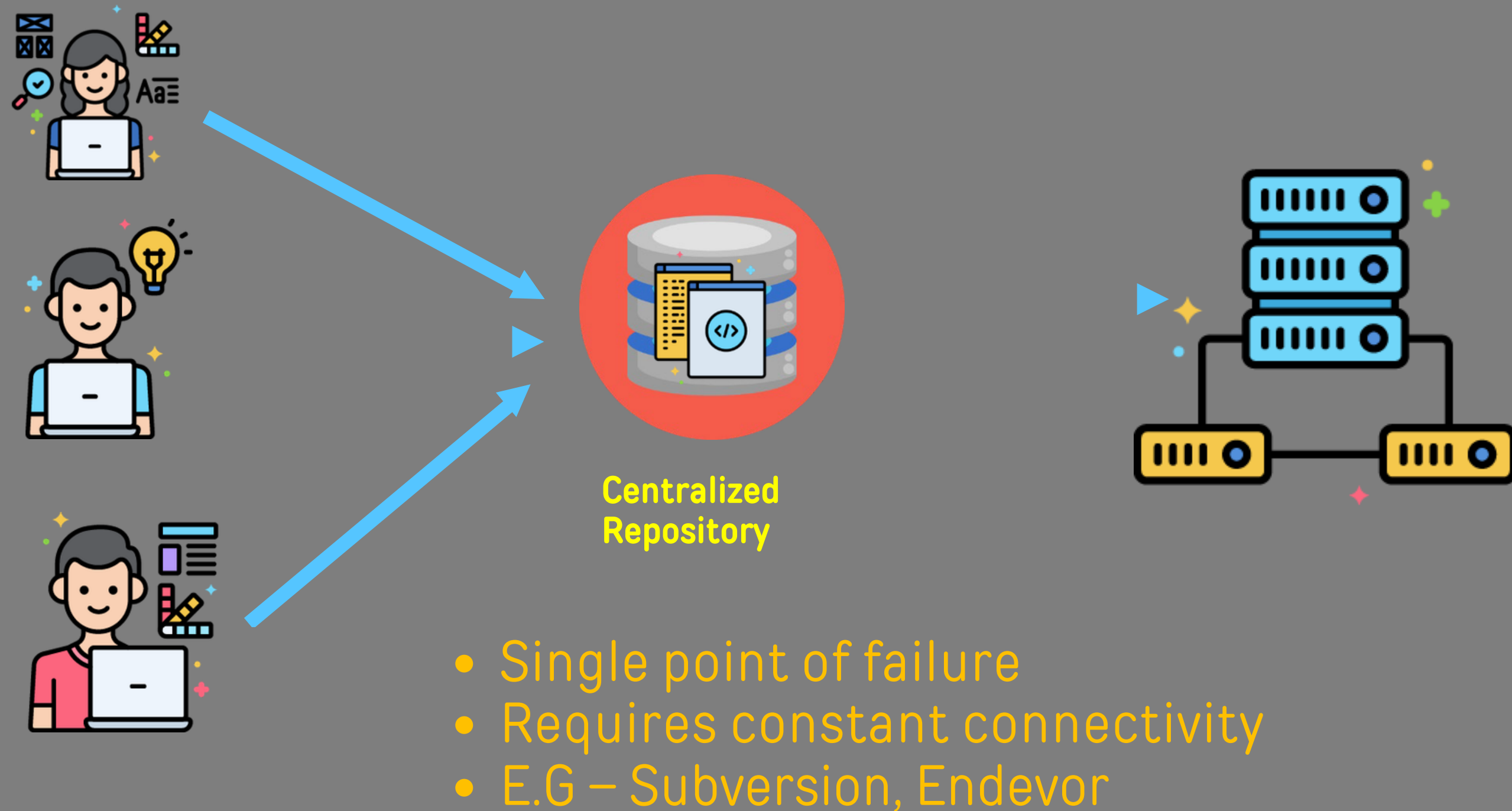
Repository

Why Git?

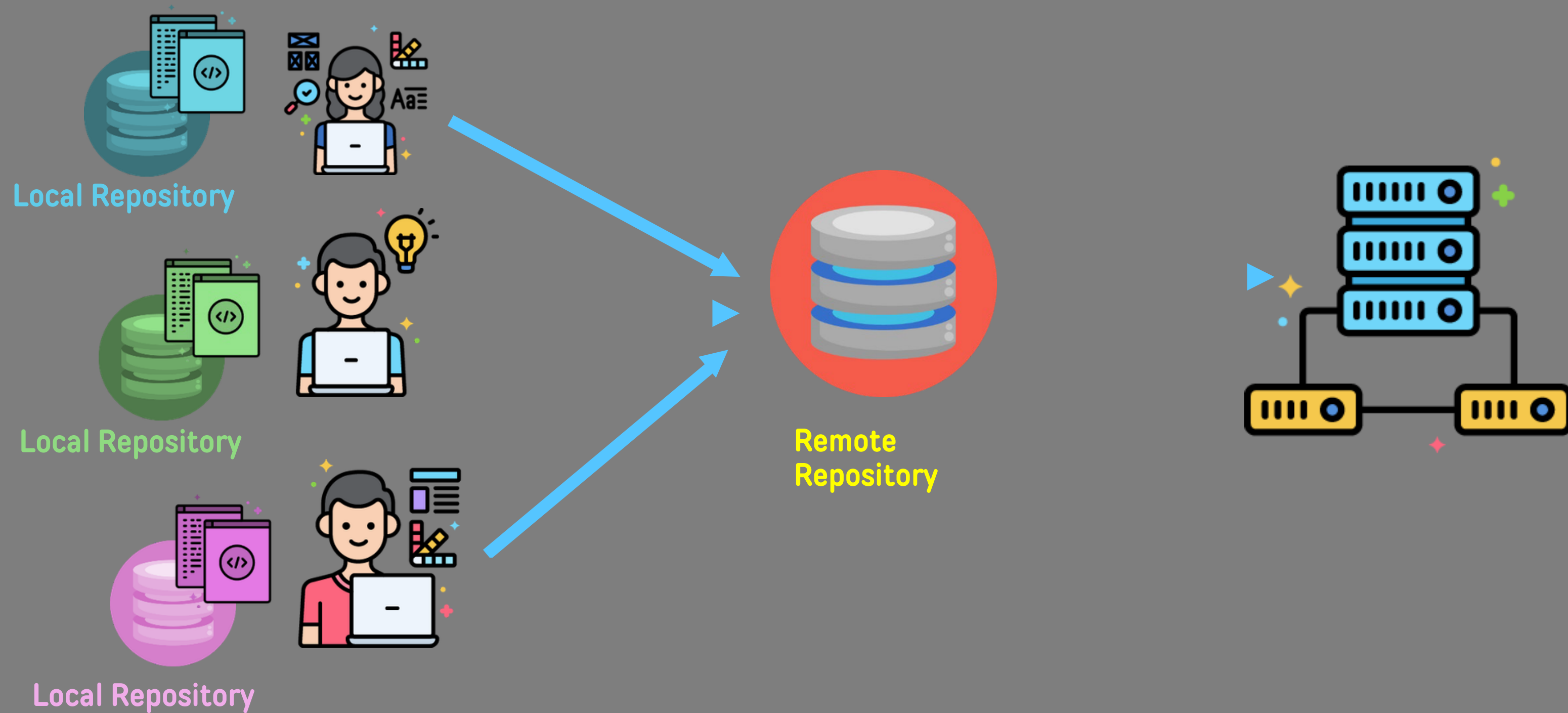
- Distributed



Centralized Version Control System



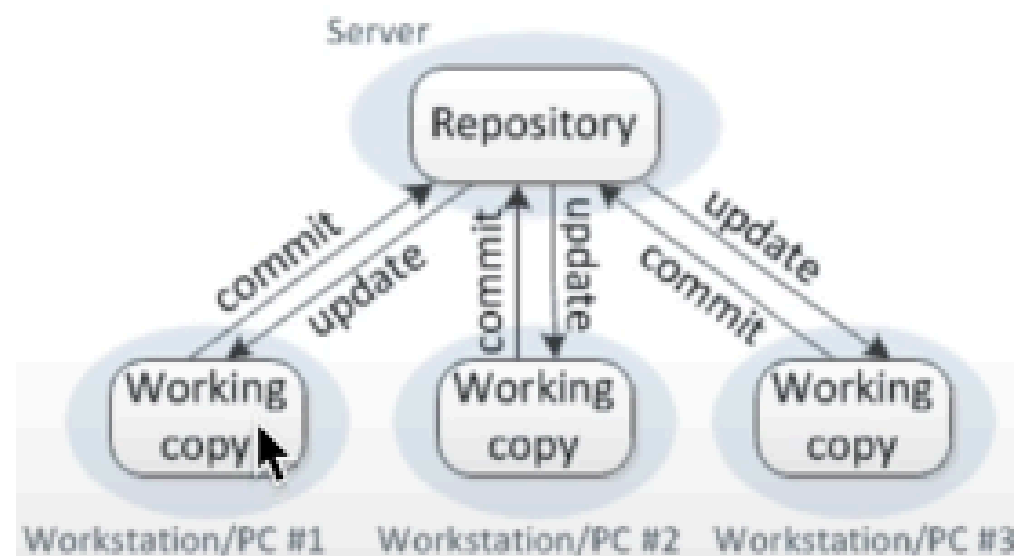
Distributed Version Control System



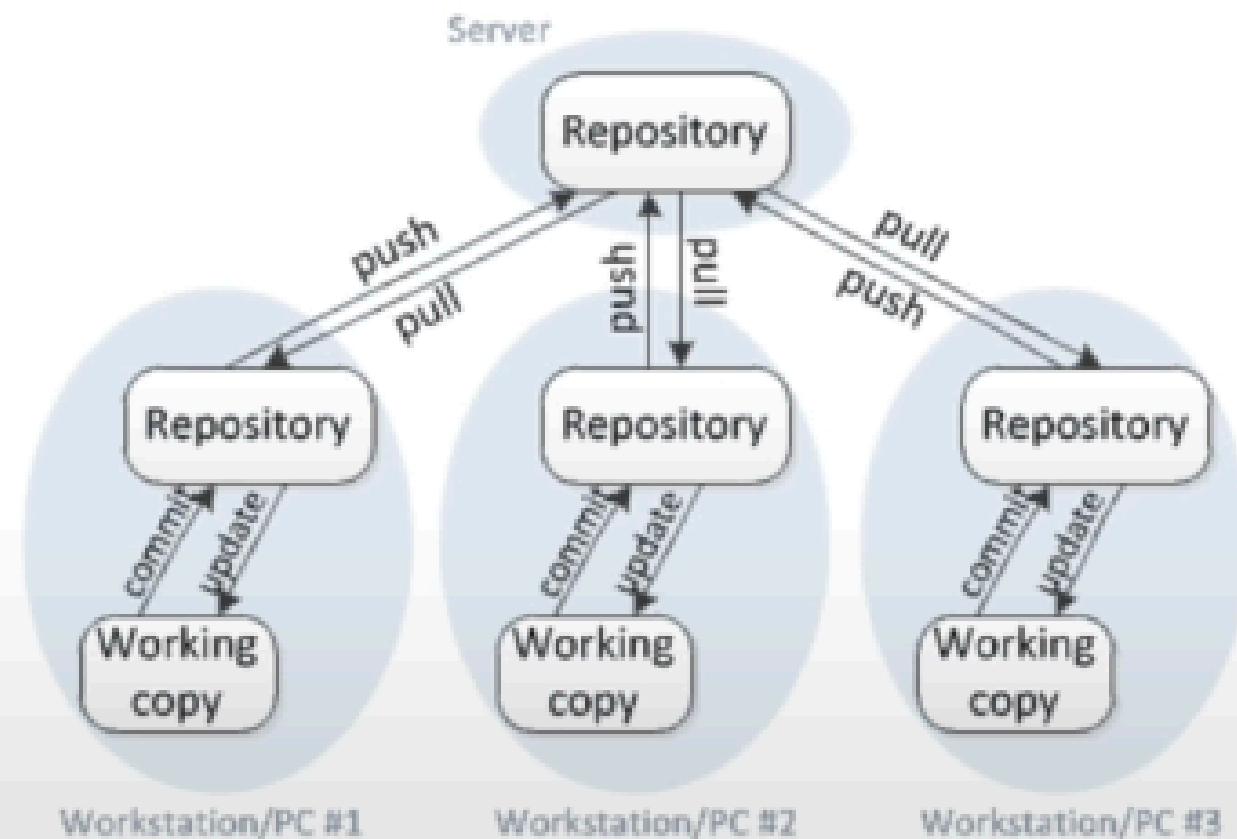
Centralized vs Distributed

Version Control

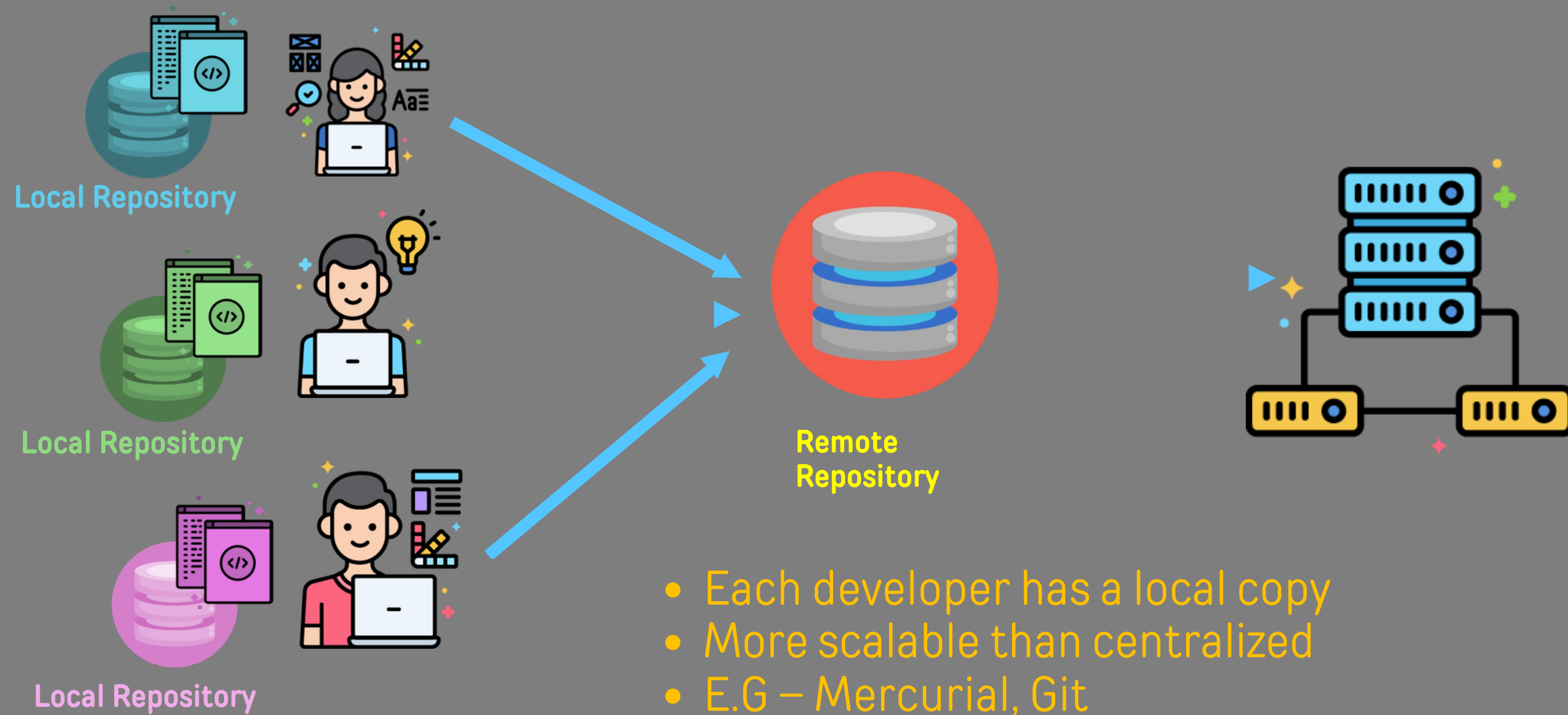
Centralized version control



Distributed version control



Distributed Version Control System



Version Control System



Why Git?

- Distributed
- Performant
- Detailed audit tracking
- Open source
 - Free!
 - Implemented with Kubernetes GitOps, integration with Jenkins and other DevOps tools
 - GitHub, GitLab, Code Commit are all based on Git

Git vs GitHub

Git Vs. GitHub

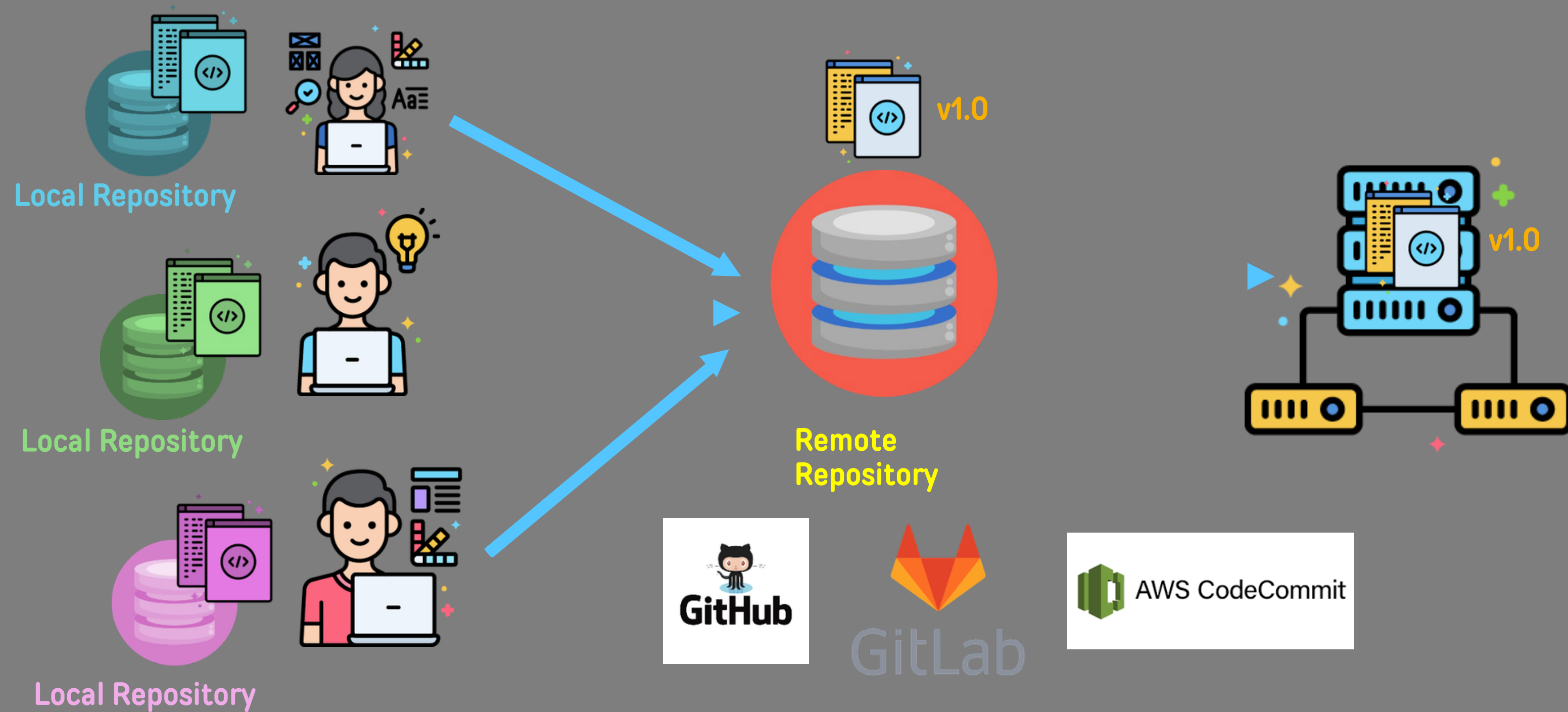


- Version Control System
- Installed locally on the system
- Created in 2005, by Linus Torvalds
- Open source, and used in multiple cloud repository services



- Git repository hosting services with other features
- Runs on the cloud
- Created in 2008, currently owned by Microsoft
- Not open source, have free and paid tiers

Distributed Version Control System



Configure Git

การตั้งค่า Git (Configure)

- ทำไมต้องตั้งค่าก่อนใช้งาน?

ทุกครั้งที่เรา commit — Git จะ **แนบชื่อ (user.name) และอีเมล (user.email)** ของผู้เขียนติดไปกับ commit นั้นถาวร เพื่อบอกว่า "ใครเป็นคนแก้โค้ดชุดนี้" ถ้ายังไม่ตั้งค่า Git จะไม่ยอมให้ commit และแจ้ง error ว่า `Please tell me who you are`



ระบุตัวตนผู้เขียน

ชื่อ + อีเมลถูกฝังในทุก commit ทำให้ตามได้ว่าใครทำอะไร เมื่อไหร่ (ดูได้จาก `git log`)



เชื่อมกับ GitHub

ถ้าอีเมลตรงกับที่ผูกไว้ใน GitHub account — commit จะโชว์รูปโปรไฟล์และนับเป็น contribution (ช่องเขียว) ของเรา



ตั้งครั้งเดียวพอ

ค่าแบบ `--global` ตั้งครั้งเดียวใช้ได้กับทุก repo บนเครื่อง ไม่ต้องตั้งใหม่ทุกโปรเจกต์

- ขั้นที่ 1 — ตั้งชื่อและอีเมล (บังคับ)

```
# ตั้งค่าชื่อและอีเมล (--global = ใช้กับทุก repo บนเครื่อง)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# ตัวอย่างจริง (สมมติ)
git config --global user.name "somchai-dev"
git config --global user.email "somchai.dev@example.com"
```

• ขั้นที่ 2 — ตั้งค่าเสริมที่แนะนำ (Optional แต่ควรทำ)

ให้ branch แรกชื่อ main (แทน master) ตรงกับมาตรฐาน GitHub

```
git config --global init.defaultBranch main
```

ตั้ง VS Code เป็น editor เวลา Git ต้องให้เราพิมพ์ข้อความ (เช่น commit message)

```
git config --global core.editor "code --wait"
```

เปิดสีในผลลัพธ์คำสั่ง อ่าน git status / git diff ง่ายขึ้น

```
git config --global color.ui auto
```

จำรหัสผ่าน/token ไว้ ไม่ต้องกรอกซ้ำทุกครั้งที่ push/pull

```
git config --global credential.helper store
```

• ขั้นที่ 3 — ตรวจสอบการตั้งค่า

ดูค่า config ทั้งหมด (กด q เพื่อออก)

```
git config --list # หรือย่อเป็น git config -l
```

```
>> user.name=somchai-dev
```

```
>> user.email=somchai.dev@example.com
```

```
>> init.defaultbranch=main
```

```
>> color.ui=auto
```

ดูทีละค่า — เช็กเฉพาะตัวที่สงสัย

```
git config user.name
```

```
>> somchai-dev
```

```
git config user.email
```

```
>> somchai.dev@example.com
```

ดูว่าค่าแต่ละตัวถูกตั้งจากไฟล์ไหน (มีประโยชน์มากเวลาค่า local ทับ global)

```
git config --list --show-origin
```

```
>> file:/home/somchai/.gitconfig user.name=somchai-dev
```

```
>> file:./git/config user.name=work-account
```

- **สลับไปใช้ GitHub อีก account**

กรณีมีหลาย account (เช่น account ส่วนตัว กับ account ของมหาวิทยาลัย) ต้องเปลี่ยนทั้ง **ตัวตนใน commit** (user.name / user.email) และ **ตัวตนที่ใช้ login** (credential.username):

```
# เปลี่ยนตัวตนที่แนบใน commit
git config --global user.name "somchai-uni"
git config --global user.email "somchai.j@university-example.ac.th"

# เปลี่ยน username ที่ใช้ยืนยันตัวตนกับ GitHub ตอน push/pull
git config --global credential.username "somchai-uni"
```

- **แก้ปัญหา: commit / push ไม่ได้เพราะค้าง account เก่า**

อาการที่พบบ่อยในห้อง Lab: เครื่องเคย login ด้วย account ของคนก่อนหน้านี้ ทำให้ push แล้วขึ้น `Permission denied` หรือ 403 — วิธีแก้คือล้าง config เก่าออกก่อน แล้วตั้งใหม่:

```
# 1) ลบ config ตัวตนเก่าออกทั้งหมด (--unset = ลบค่านั้นทิ้ง)
git config --global --unset user.name
git config --global --unset user.email
git config --global --unset credential.username

# 2) ถ้า repo ชี้ remote ของคนเก่าอยู่ ให้ถอด origin ออก
git remote -v          # เช็คก่อนว่า origin ชี้ไปที่ไหน
>> origin https://github.com/old-user/old-repo.git (fetch)
git remote remove origin  # ลบ origin เก่า แล้วค่อย git remote add ใหม่

# 3) ตั้งค่า account ของเราใหม่ตามขั้นที่ 1
```

Creating a Git Repository

Day 1 - Starting with Git

- How can we create a Git Repository?
 - **git init**
 - This command initializes a Git Repository on your local machine.
 - You only need to run this command once per project.
 - **git status**
 - This command will report back the status of your Git repository.

Day 1 - Starting with Git

- How can we create a Git Repository?
 - Upon creating a repository with **git init** you will create a hidden .git file.
 - The .git file is a hidden file that manages the versioning of the files inside the Git repository.

Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.

Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



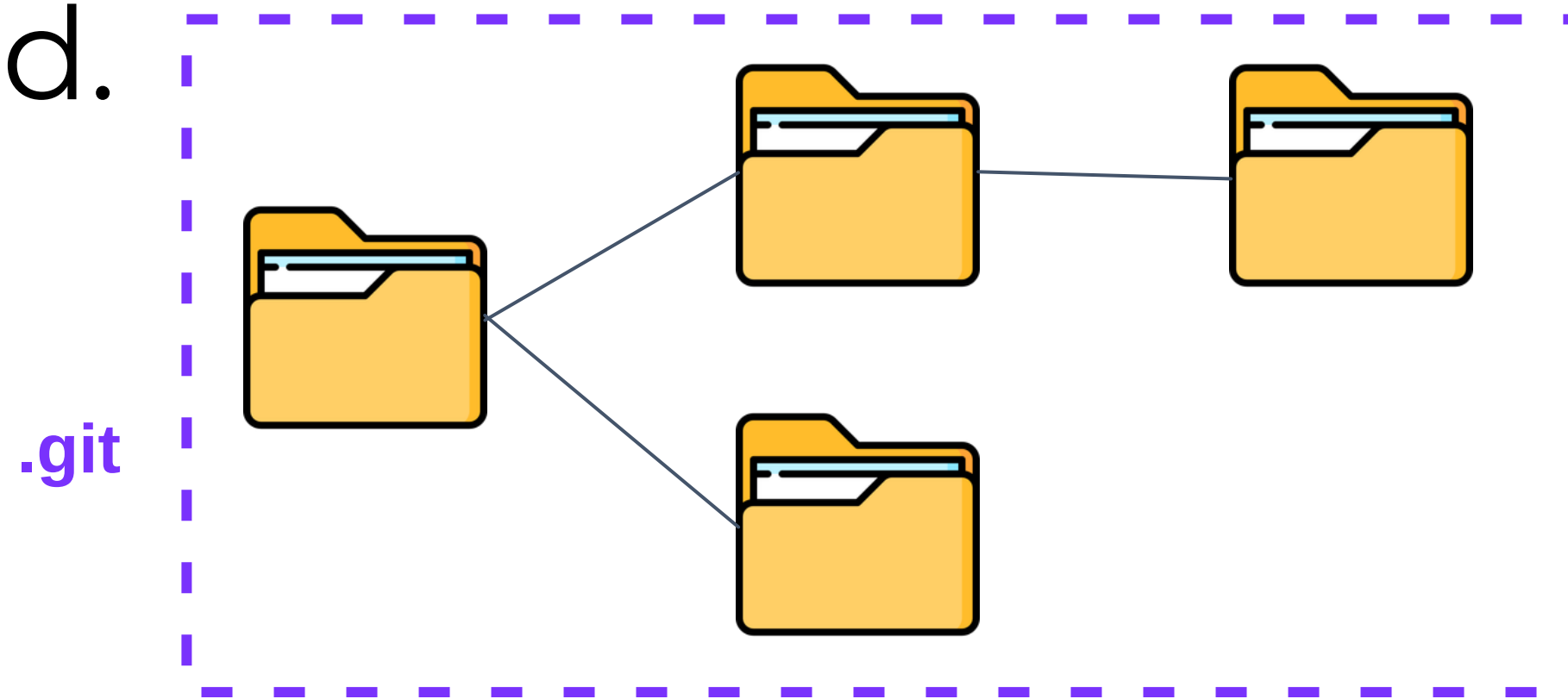
Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



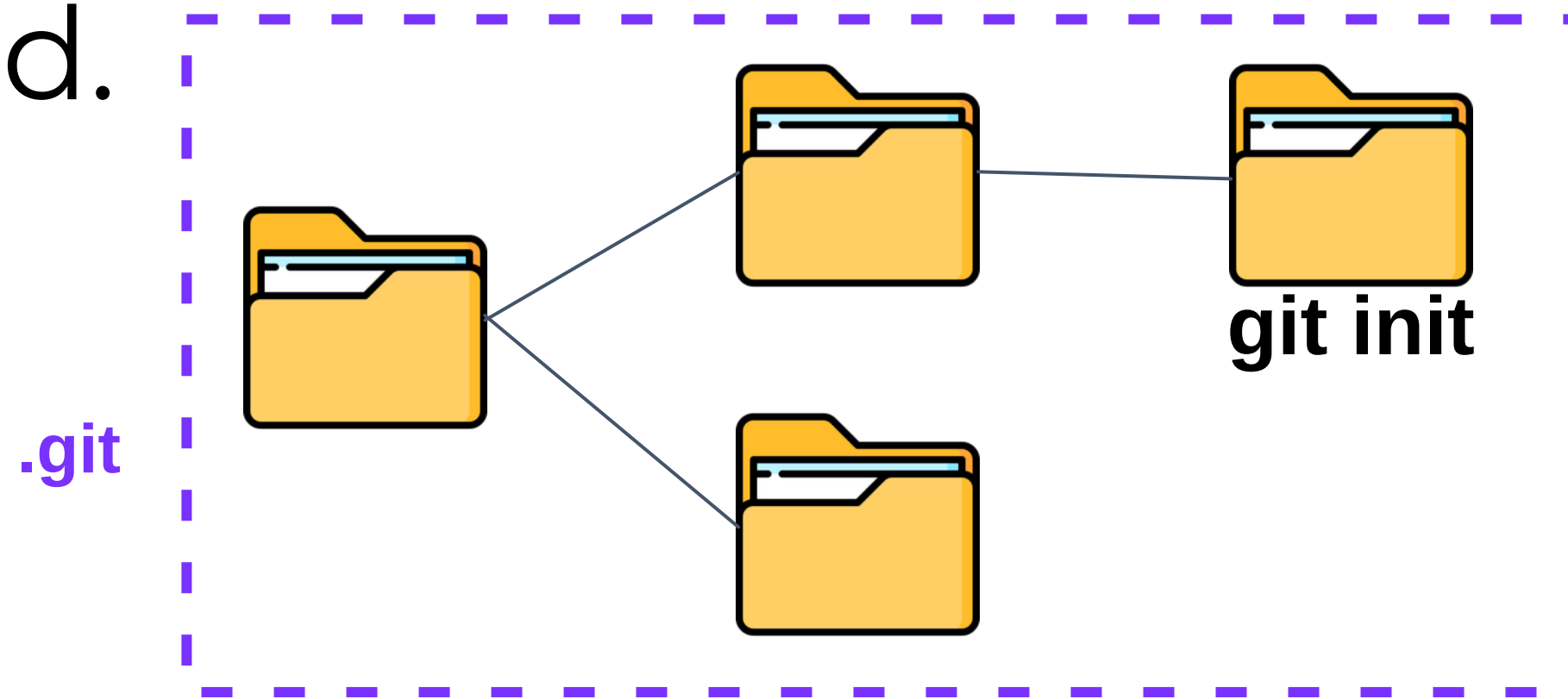
Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



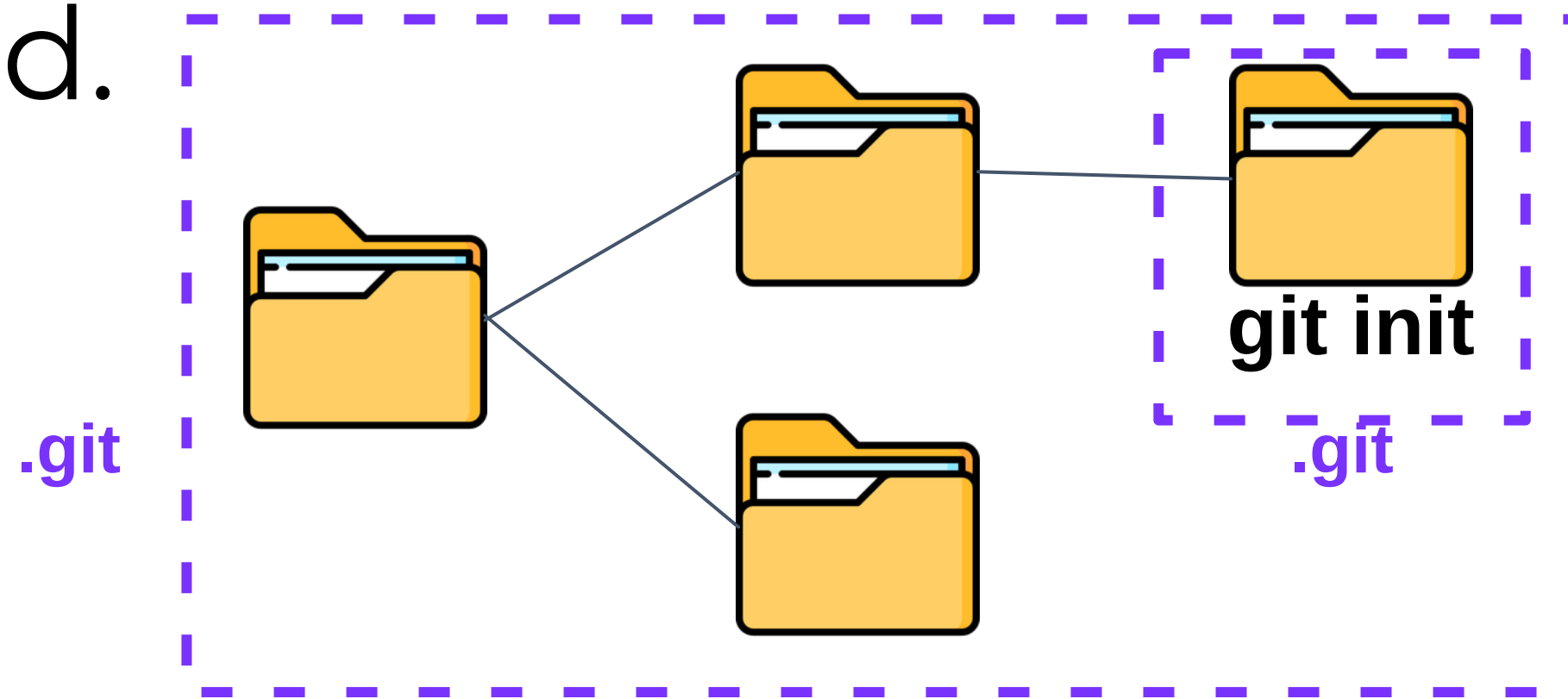
Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



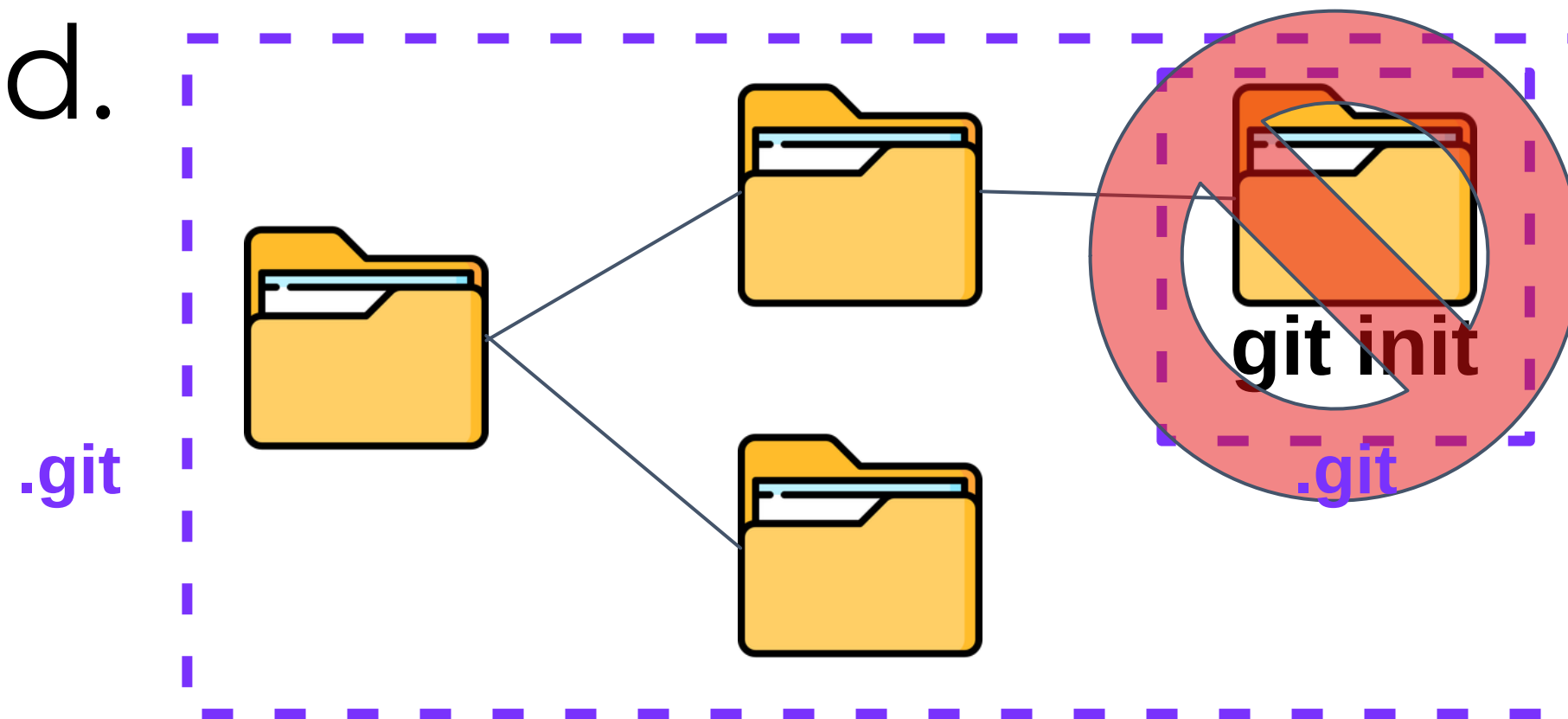
Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



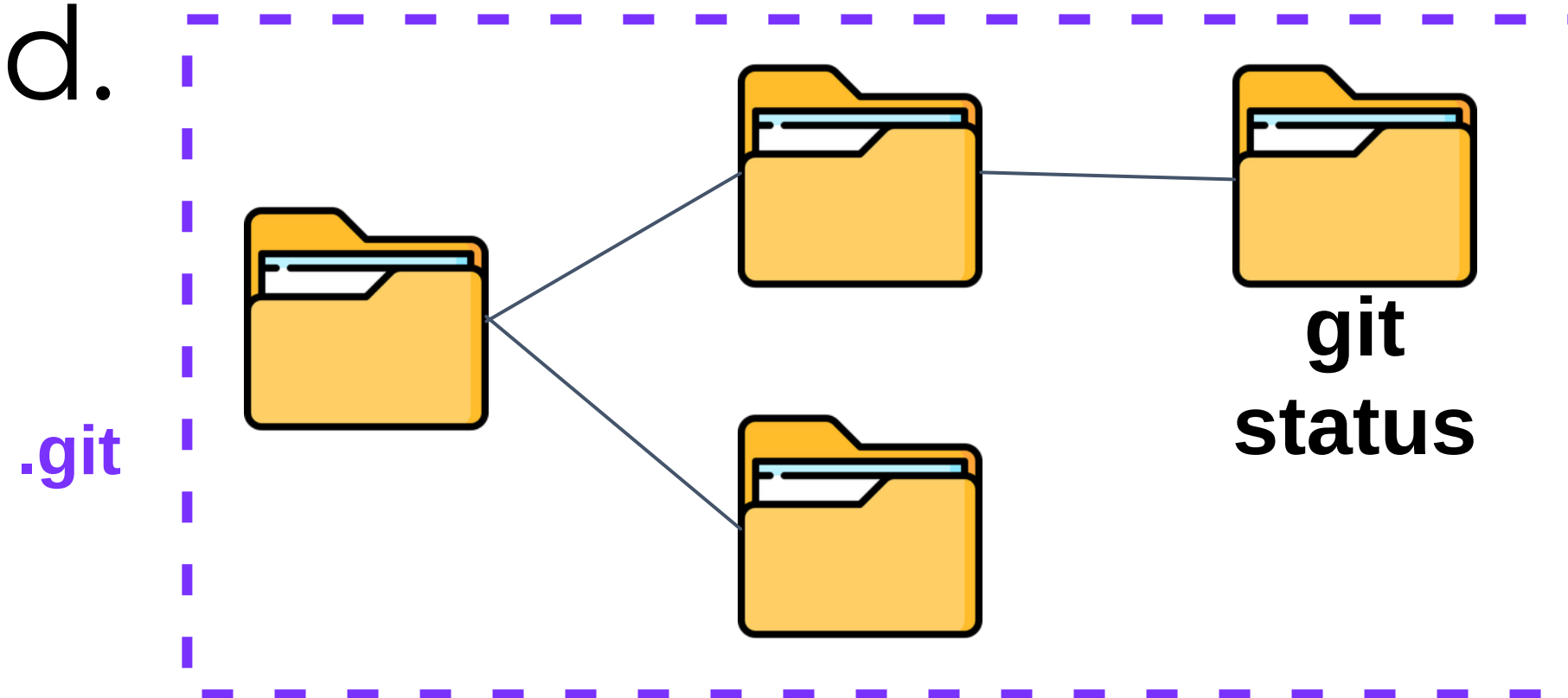
Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



Day 1 - Starting with Git

- Git inside a Folder/Directory:
 - Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



Ignoring Files

We can tell Git which files and directories to ignore in a given repository, using a `.gitignore` file. This is useful for files you know you **NEVER** want to commit, including:

- Secrets, API keys, credentials, etc.
- Operating System files (`.DS_Store` on Mac)
- Log files
- Dependencies & packages



.gitignore

Create a file called .gitignore in the root of a repository. Inside the file, we can write patterns to tell Git which files & folders to ignore:

`.DS_Store` will ignore files named .DS_Store

`folderName/` will ignore an entire directory

`*.log` will ignore any files with the .log extension

<https://www.toptal.com/developers/gitignore>



gitignore.io

สร้างไฟล์ .gitignore ที่มีประโยชน์สำหรับโปรเจกต์ของคุณ

ค้นหาแบบปฏิบัติการ IDE หรือภาษาการเขียนโปรแกรม

สร้าง

ชอร์สโค้ด | ลองอ่านเอกสาร คำสั่งของ คอมมิตไลน์ ดูลิ!



Private Repositories and Tokens

Day 1 - Starting with Git

- Clone Syntax with PAT:

git clone https://username:YOUR_TOKEN@github.com/username/repo.git

- Previously we used:

git clone https://github.com/account/repo.git

Create a Personal Access Token

Your personal account

Public profile

Account

Appearance

Accessibility

Notifications

ss

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Enterprises

Moderation

, planning, and automation

Repositories

Codespaces

Packages

Copilot

Pages

Saved replies

Privacy

Code security and analysis

Integrations

Applications

Scheduled reminders

Notes

Security log

Sponsorship log

Developer settings

Public profile

Name

tuchsanai

Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.

Public email

Select a verified email to display

You have set your email address to private. To toggle email privacy, go to [email settings](#) and uncheck "Keep my email address private."

Bio

Tell us a little bit about yourself

You can @mention other users and organizations to link to them.

Pronouns

Don't specify

URL

Social accounts

Link to social profile

Link to social profile

Link to social profile

Link to social profile

Company

You can @mention your company's GitHub organization to link to it.

Location

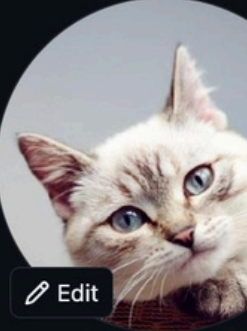
Display current local time

Other users will see the time difference from their local time.

the fields on this page are optional and can be deleted at any time, and by filling them out, you're giving us consent to share this data wherever your user profile appears. Please see our [privacy statement](#) to learn more about how we use this information.

Update profile

Profile picture



Edit

In the left sidebar, select "Developer settings."

Settings / Developer Settings

Type to search

GitHub Apps

OAuth Apps

Personal access tokens

GitHub Apps

New GitHub App

Want to build something that integrates with and extends GitHub? [Register a new GitHub App](#) to get started developing on the GitHub API. You can also read more about building GitHub Apps in our [developer documentation](#).

4

© 2023 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

Settings / Developer Settings

Type to search

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)

Generate new token

Need an API token for scripts or testing? [Generate a personal access token](#) for quick access to the GitHub API.

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

5

© 2023 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

6

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

test

What's this token for?

Expiration *

7 days

The token will expire on Tue, Sep 26 2023

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events

☒ repo

repo:status

repo_deployment

public_repo

repo:invite

security_events

Full control of private repositories

Access commit status

Access deployment status

Access public repositories

Access repository invitations

Read and write security events

☐ workflow

write:packages

read:packages

Update GitHub Action workflows

Upload packages to GitHub Package Registry

Download packages from GitHub Package Registry

☐ delete:packages

admin:org

write:org

read:org

manage_runners:org

Delete packages from GitHub Package Registry

Full control of orgs and teams, read and write org projects

Read and write org and team membership, read and write org projects

Read org and team membership, read org projects

Manage org runners and runner groups

☐ admin:public_key

write:public_key

read:public_key

Full control of user public keys

Write user public keys

Read user public keys

☒ admin:repo_hook

write:repo_hook

read:repo_hook

Full control of repository hooks

Write repository hooks

Read repository hooks

☐ admin:org_hook

Full control of organization hooks

☐ gist

Create gists

☐ notifications

Access notifications

☐ user

read:user

user:email

user:follow

Update ALL user data

Read ALL user profile data

Access user email addresses (read-only)

Follow and unfollow users

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Beta

Tokens (classic)

Personal access tokens (classic)

Generate new token ▼

Revoke all

Tokens you have generated that can be used to access the [GitHub API](#).

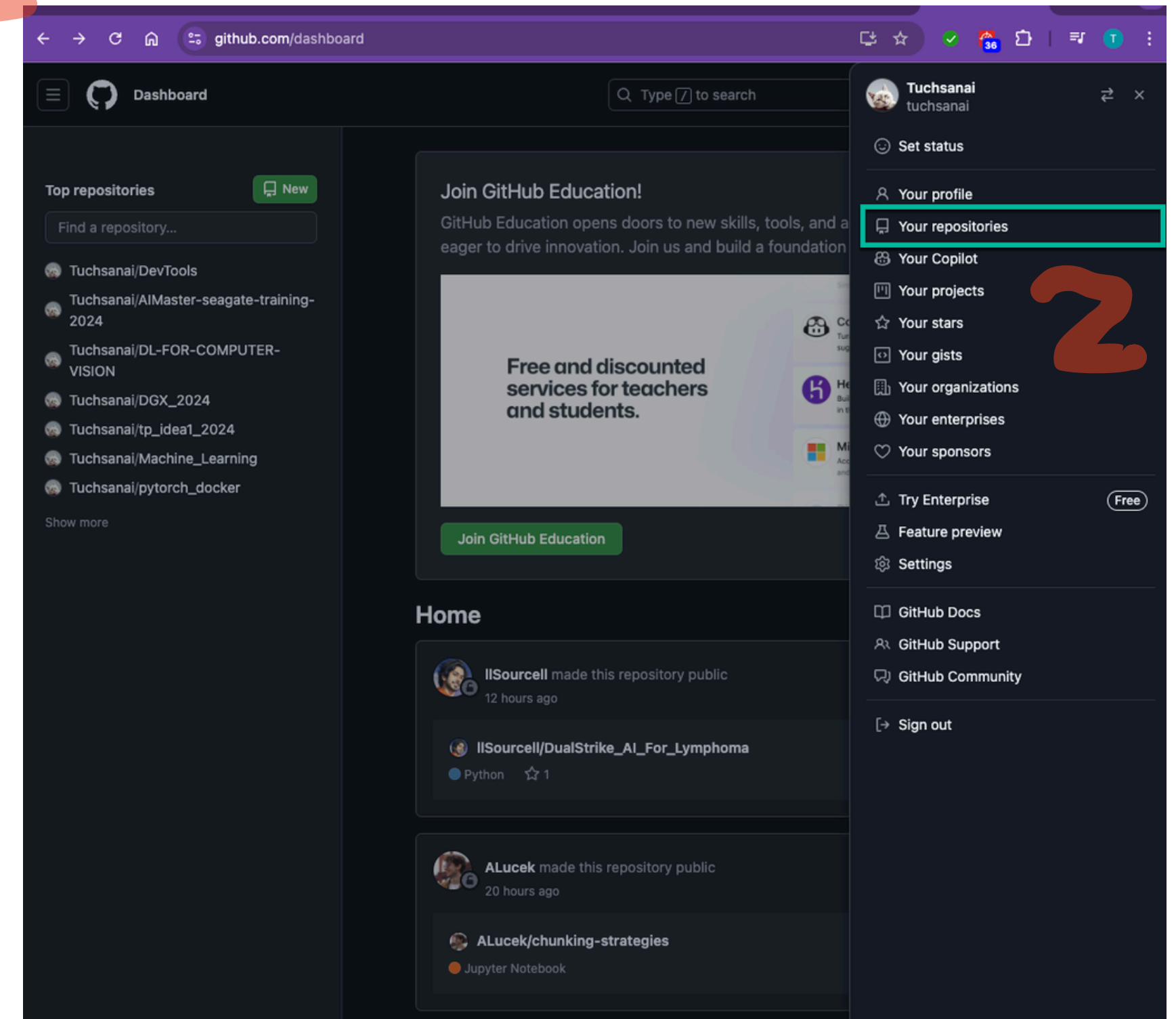
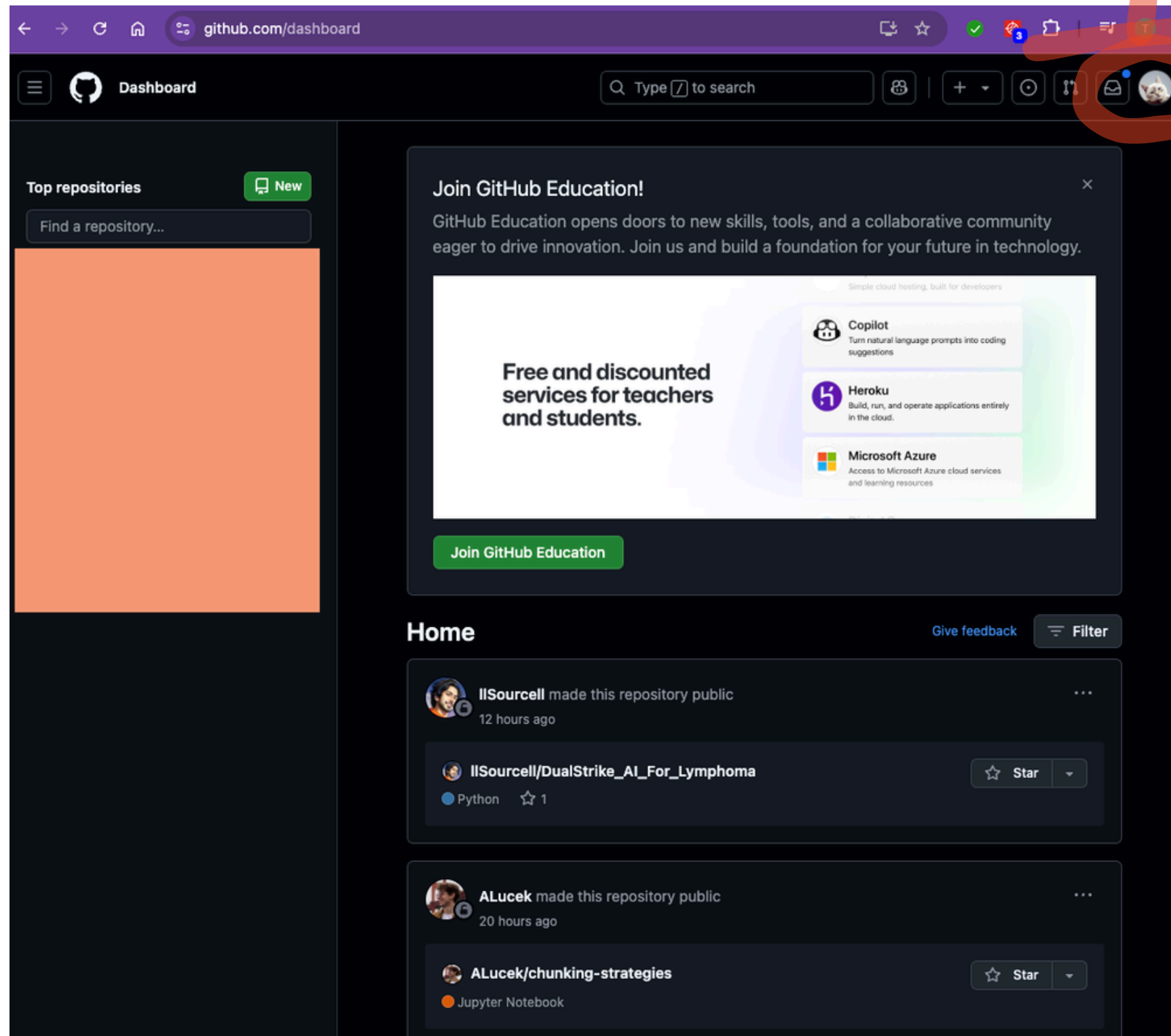
Make sure to copy your personal access token now. You won't be able to see it again!

✓ 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

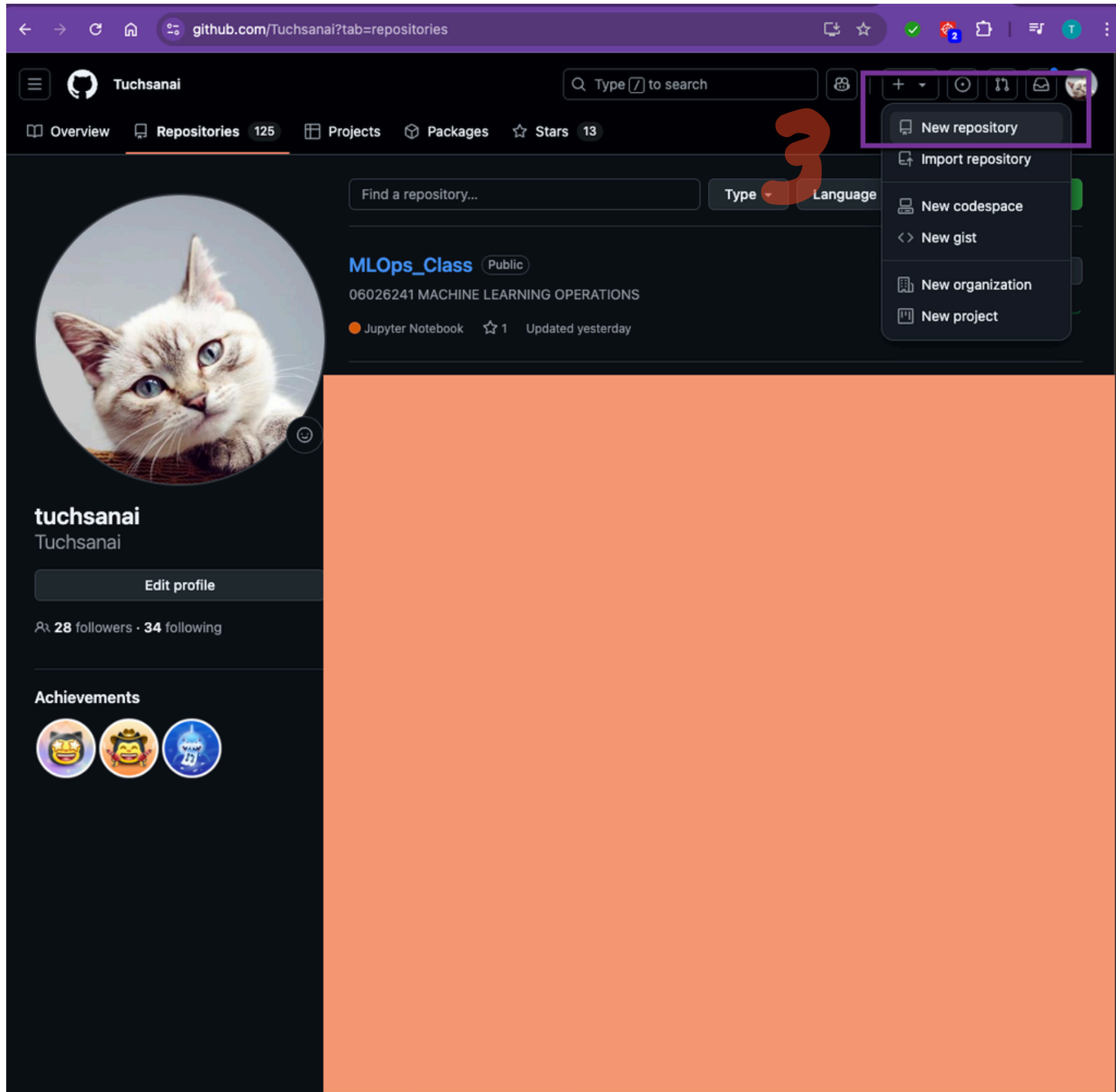
Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

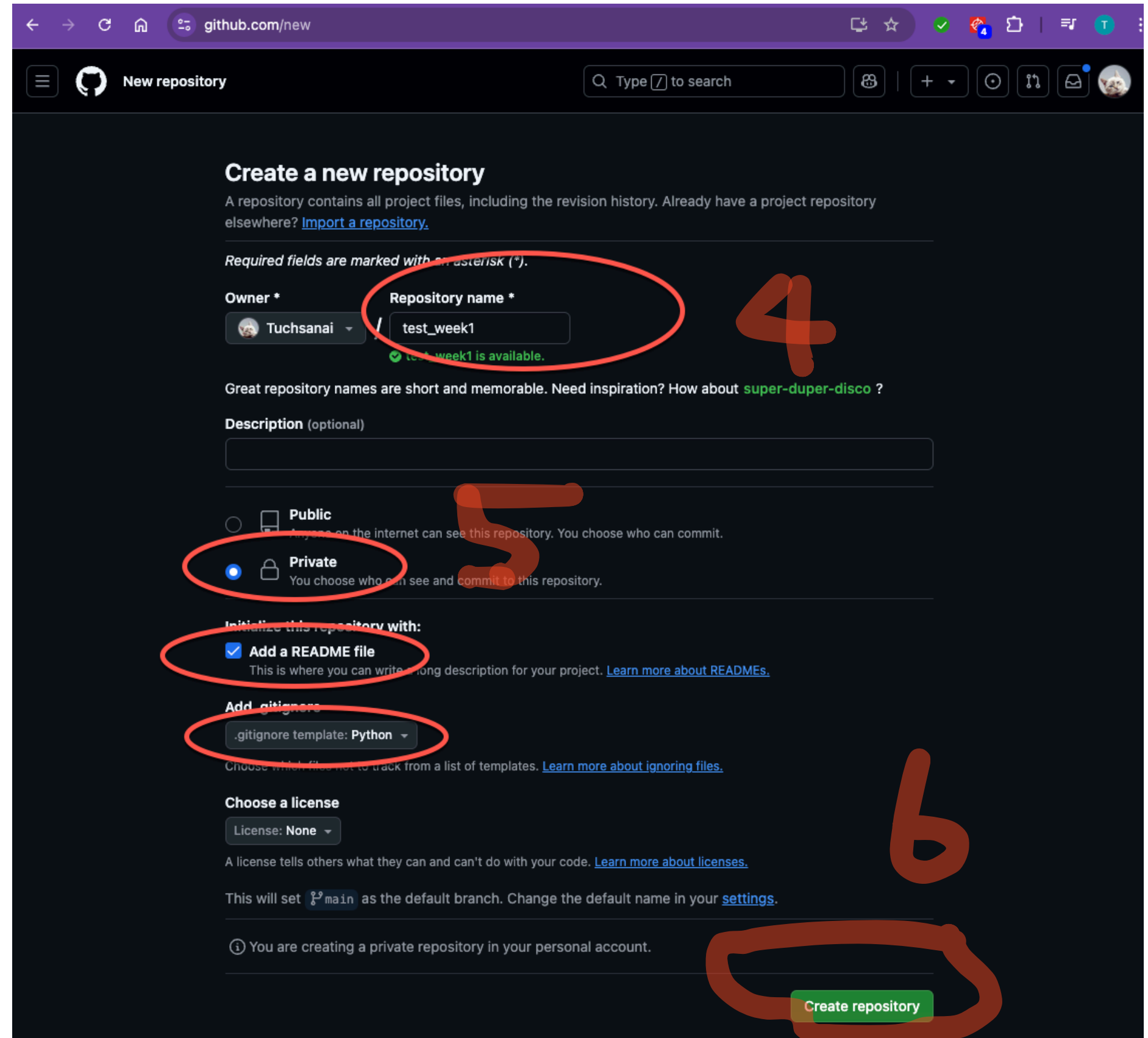
**Create a new Private
Repository**



Click New repository

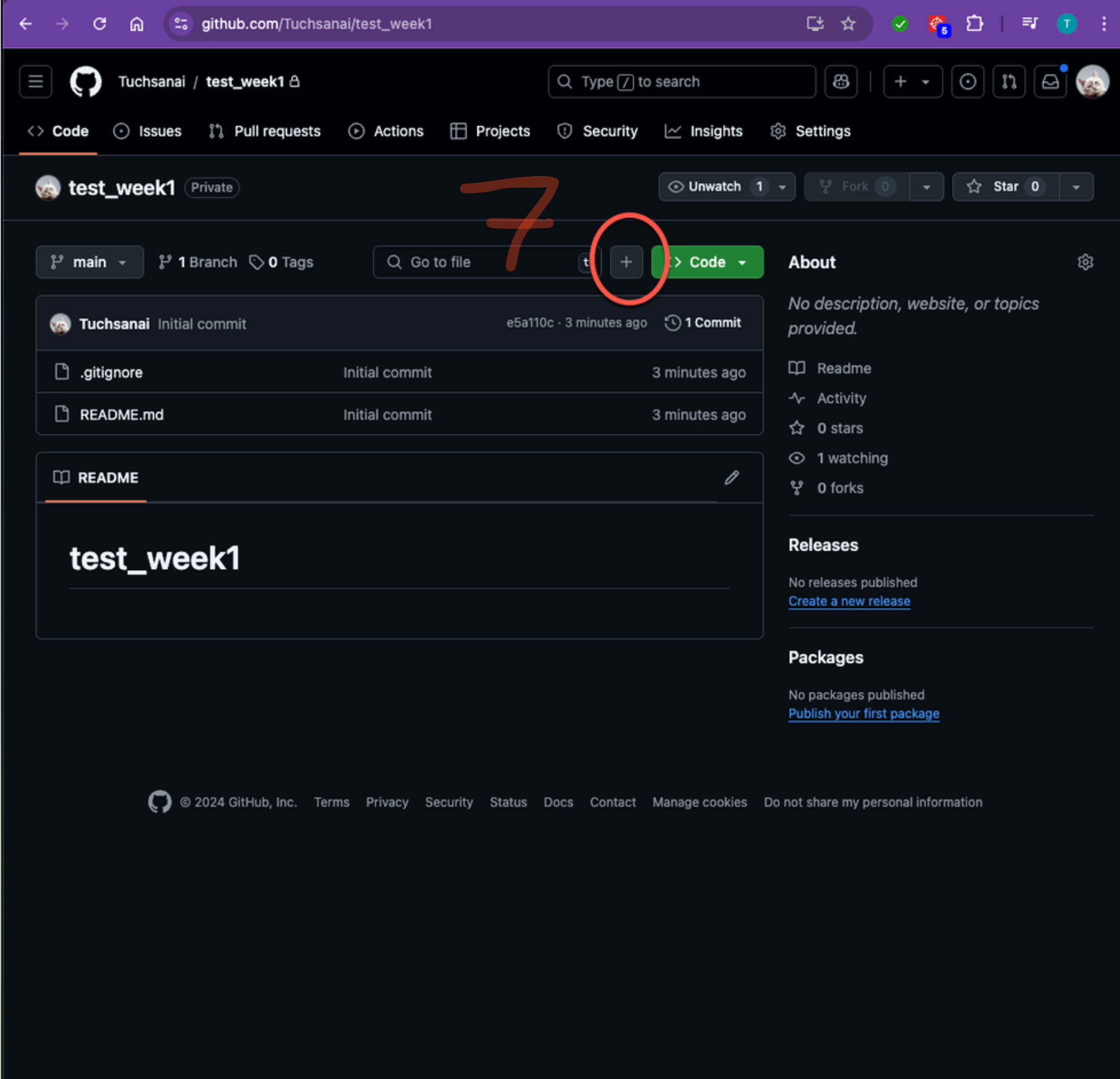
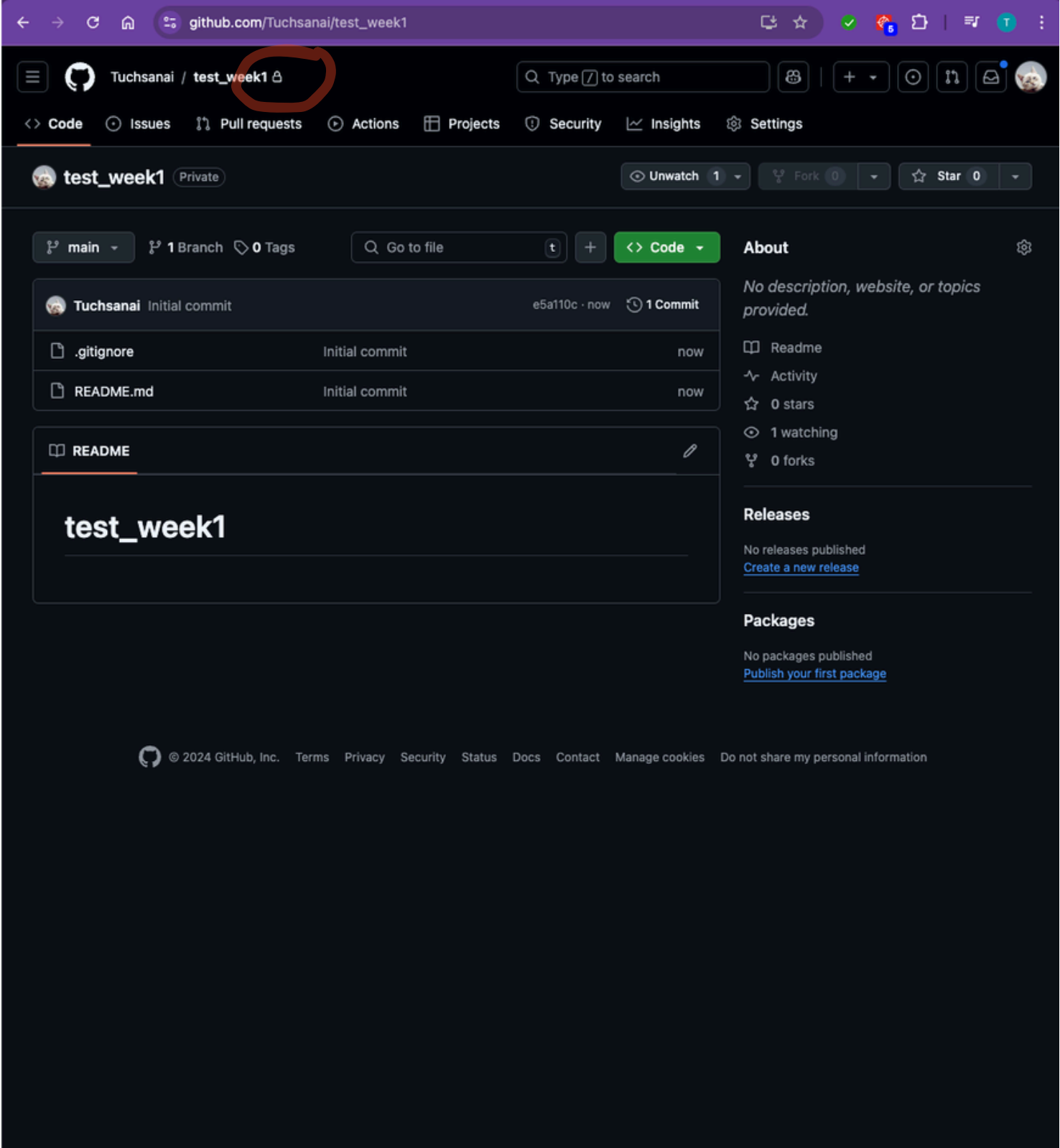


The screenshot shows the GitHub profile page for user 'tuchsanai'. The top navigation bar includes a search bar and a dropdown menu with the following options: 'New repository', 'Import repository', 'New codespace', 'New gist', 'New organization', and 'New project'. A red box highlights the 'New repository' button, and a large red number '3' is placed next to it. The profile section on the left shows the user's name, profile picture, and a list of repositories, including 'MLOps_Class'.

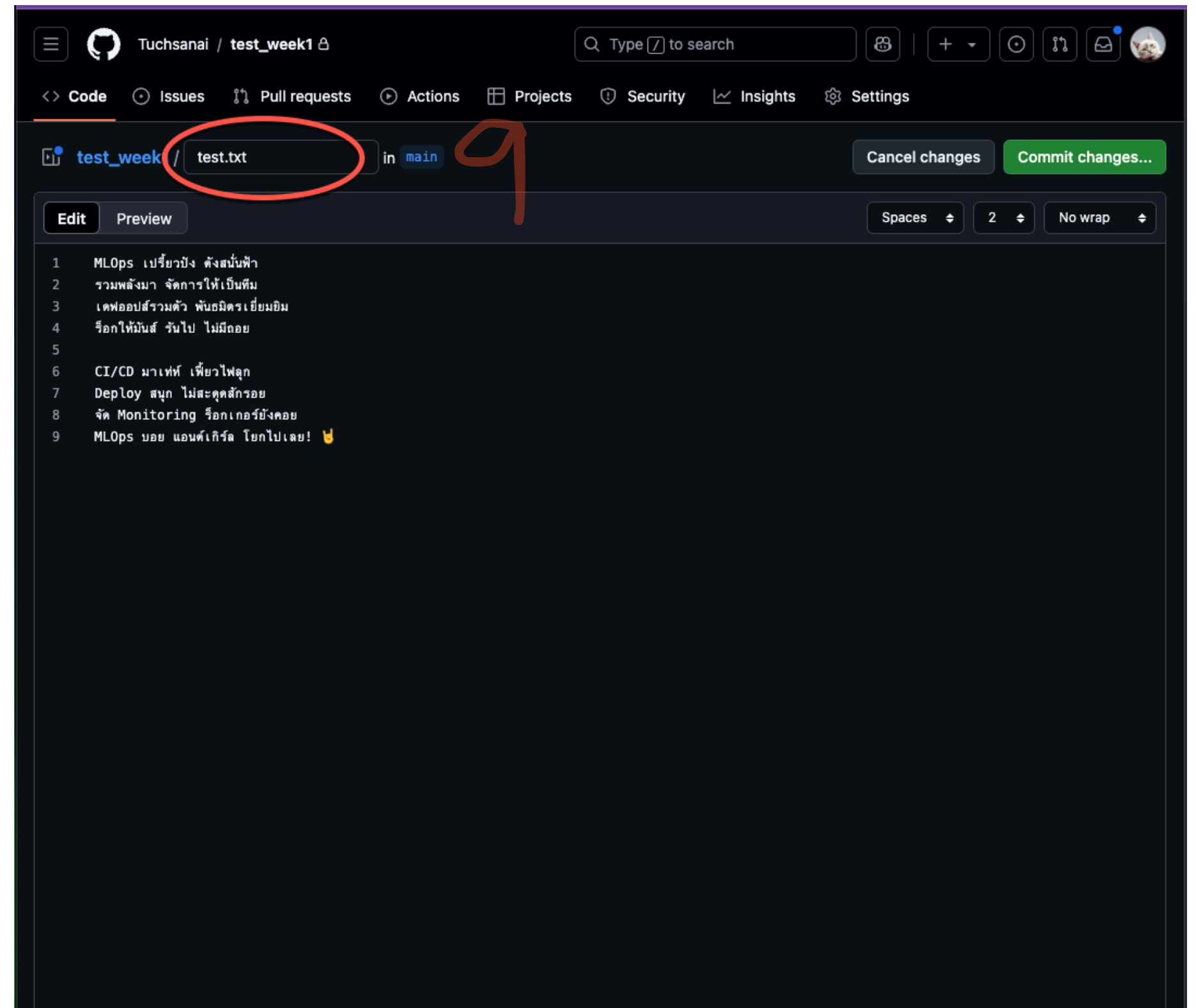
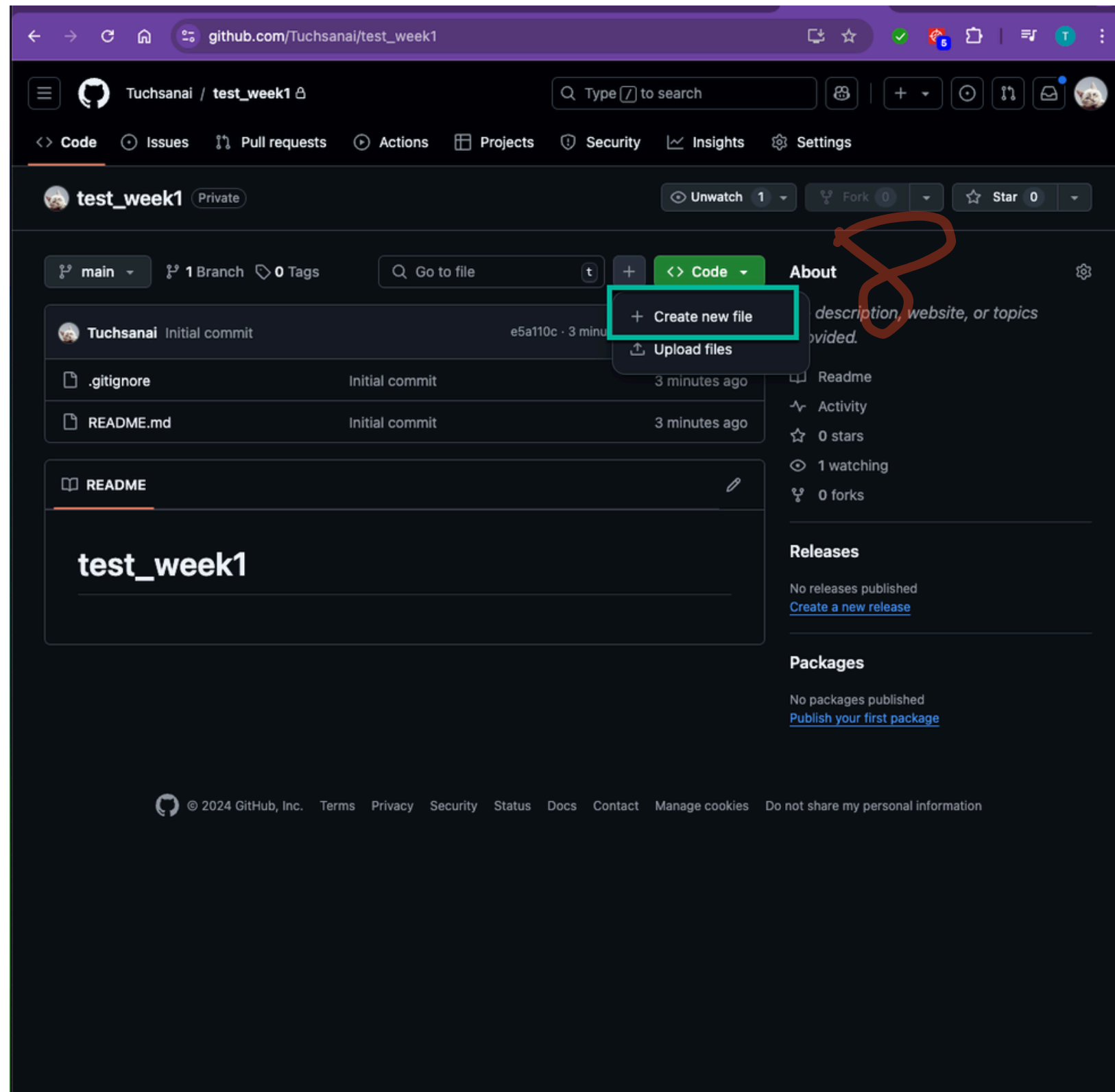


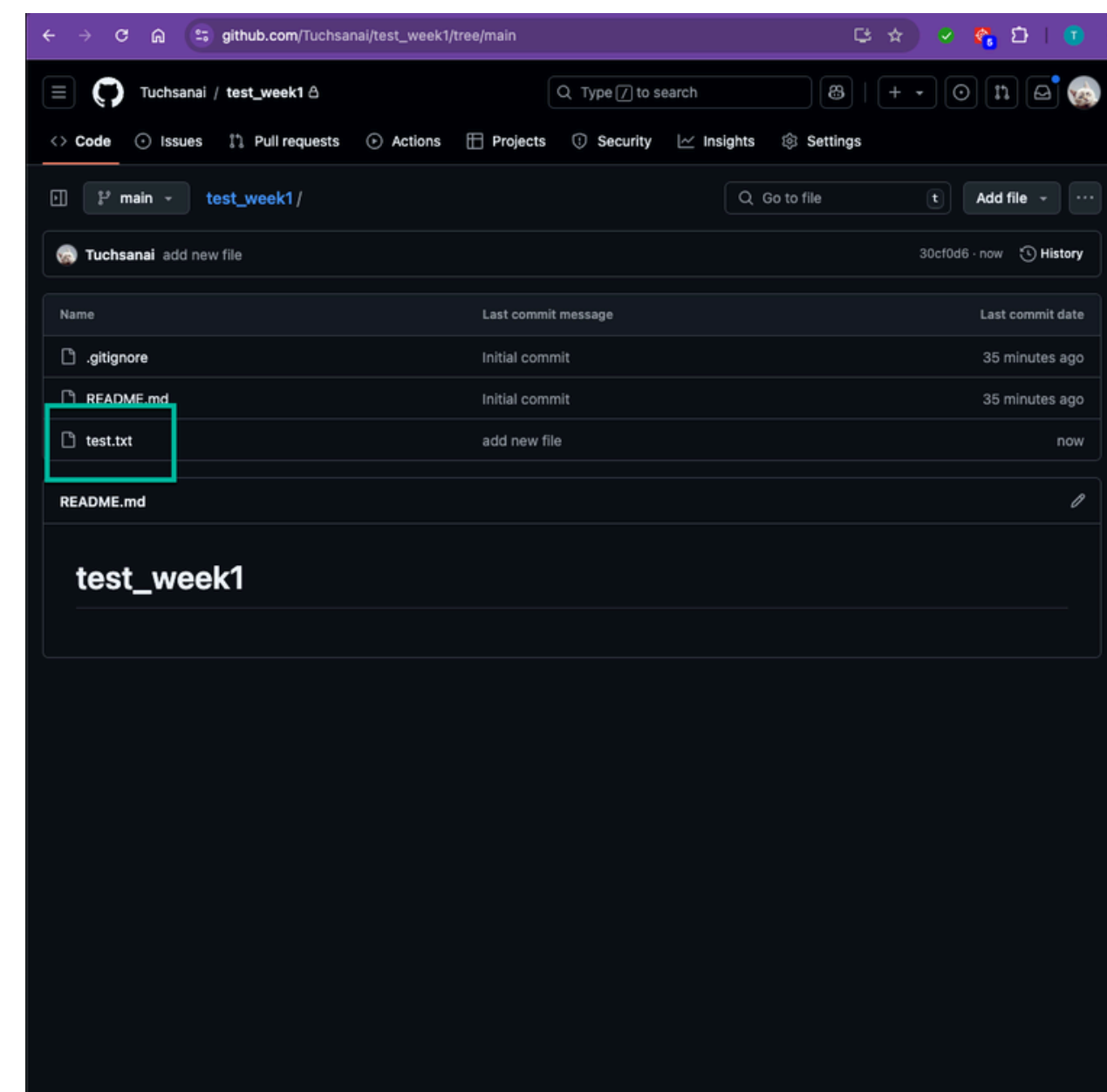
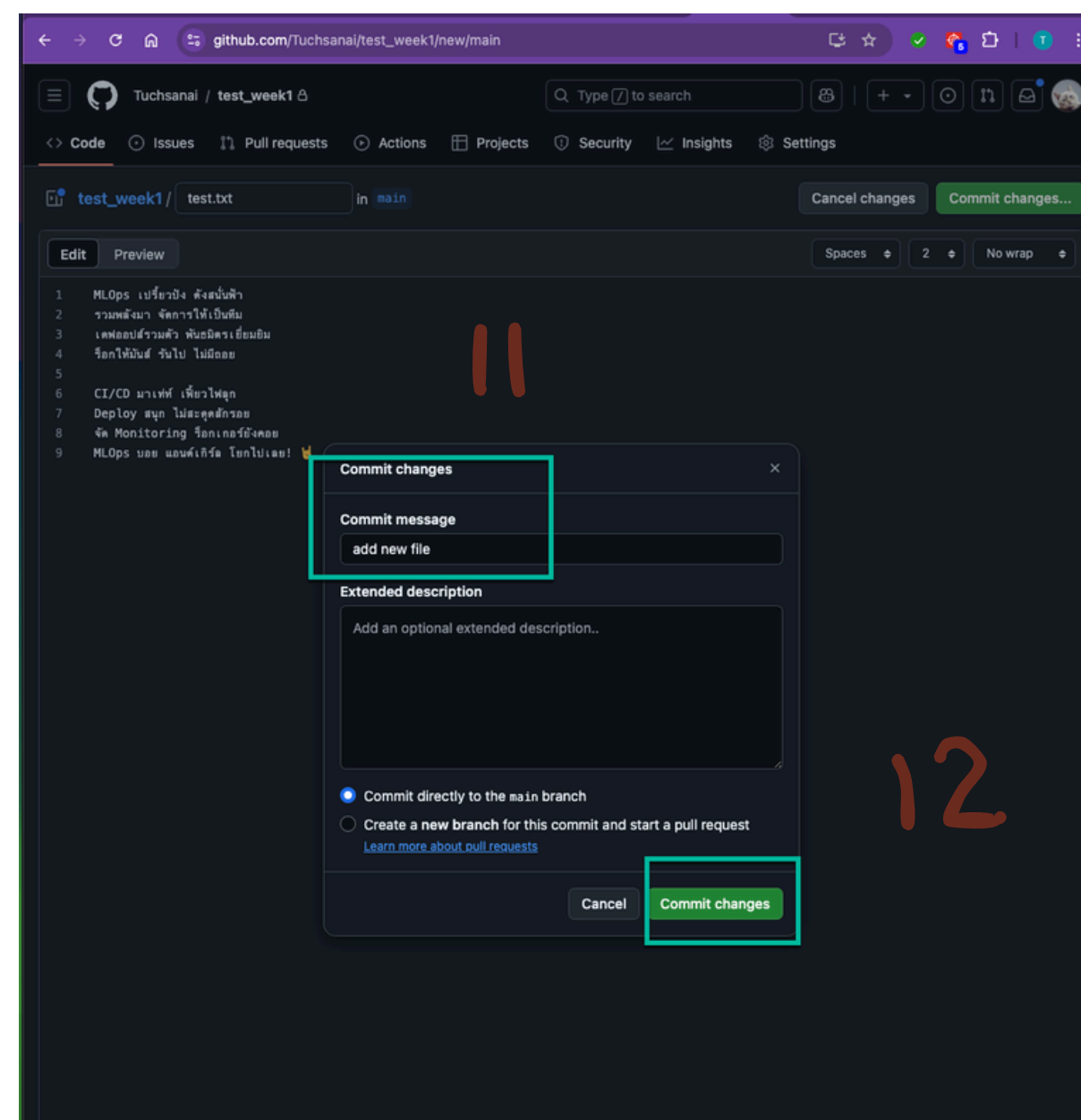
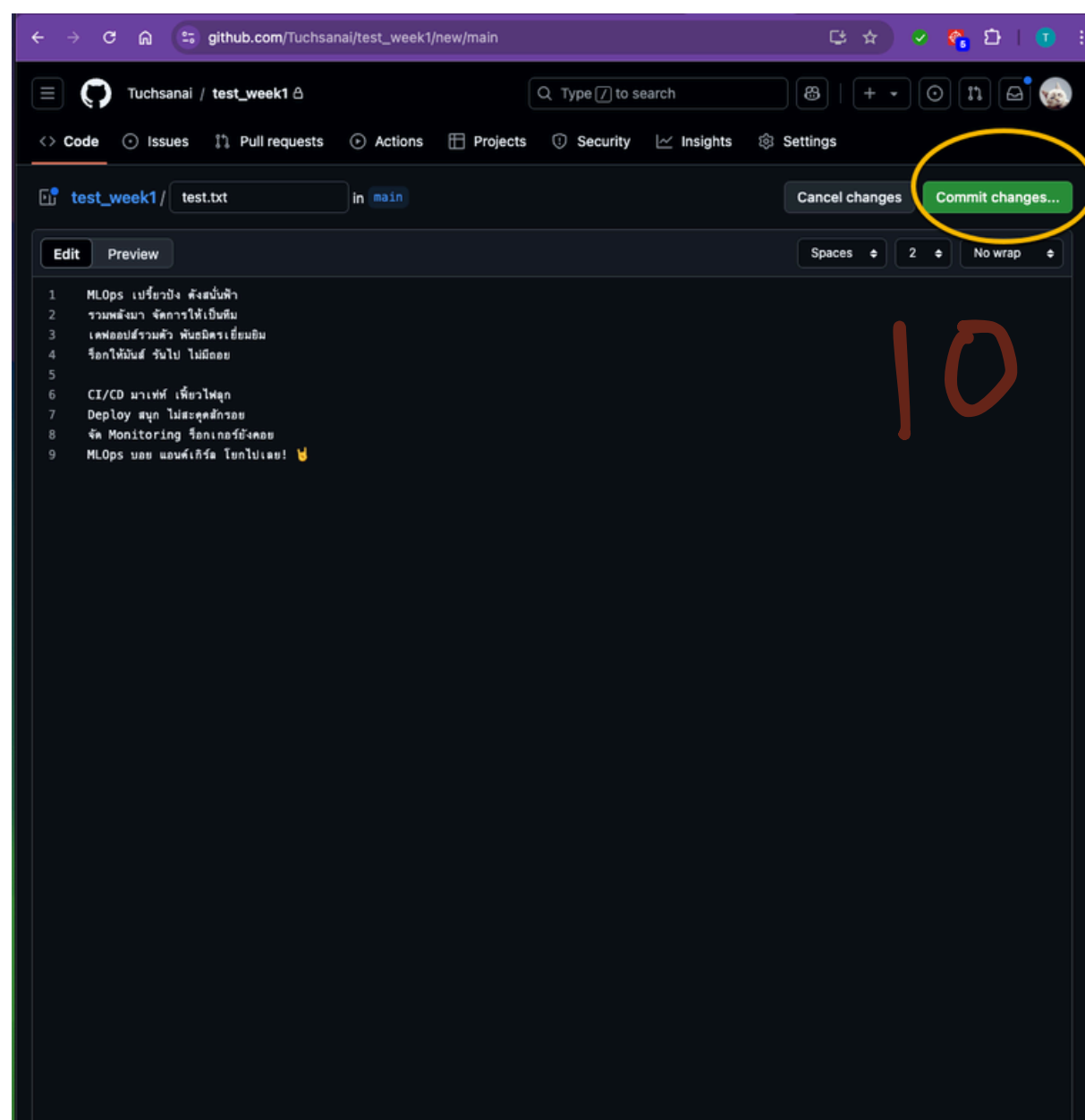
The screenshot shows the 'Create a new repository' page. The page includes a form to create a new repository. The 'Repository name' field is highlighted with a red box and a large red number '4'. The 'Private' visibility option is highlighted with a red box and a large red number '5'. The 'Add a README file' checkbox is highlighted with a red box. The 'Create repository' button is highlighted with a red box and a large red number '6'. The page also includes a description field, a dropdown for the repository's visibility (Public or Private), and a section for initializing the repository with a README file and a .gitignore template.

Privated repository



สร้าง file : test.txt





- Clone private repository:

git clone https://username:YOUR_TOKEN@github.com/username/repo.git

สรุปคำสั่งสำคัญ (Cheat Sheet)

```
# — ตรวจสอบ / ตั้งค่า —————
git --version                # เช็คว่ามี Git และเวอร์ชันอะไร
git config --global user.name "name"
git config --global user.email "email"
git config --global credential.username "user"
git config --list             # ดู config ทั้งหมด (= git config -l)
git config --list --show-origin # ดูว่าค่ามาจากไฟล์ไหน

# — ลบ config / origin เก่า —————
git config --global --unset user.name
git config --global --unset user.email
git remote remove origin

# — สร้าง / ตรวจสอบ repo —————
git init                    # สร้าง repo (เกิดโฟลเดอร์ .git)
git status                  # ดูสถานะปัจจุบัน

# — clone —————
git clone https://github.com/user/repo.git
git clone https://user:TOKEN@github.com/user/repo.git # private
```